

DATA WAREHOUSING AND BUSINESS INTELLIGENCE



Foundations: Why a Warehouse Exists

**Day 1
Foundations**

Day 1 - Concepts

DSS Evolution · OLTP vs OLAP · Warehouse vs Database

Day 1 Overview



Concepts (3 hours)

- 0:00** Orientation - tools and connectivity
- 0:15** DSS evolution; OLTP vs OLAP; Warehouse vs Database
- 1:05** *Break*
- 1:20** DBeaver guided walk: OLTP schema vs DW tables
- 2:30** Discussion: what 'decision support' means + wrap

Lab (3 hours)

- 0:00** Recap
- 0:10** Same question on OLTP vs DW - compare effort
- 1:10** *Break*
- 1:25** Manager questions → DW requirements sketch
- 2:30** Share-out; preview the star schema; wrap

Deliverable: one-page scenario + three business questions the warehouse must answer. Photographed or handed in at end.

Tool Orientation - What is Running Today



PostgreSQL *Database engine*

OLTP demo DB + empty DW schema are both loaded. Verify: can you see both in DBeaver?

DBeaver Community *SQL client + ERD viewer*

Connected to PostgreSQL. ERD auto-draw verified. This is your interface today.

Jupyter (running) *Python / pandas - Days 3-4*

Already running in the background. Note the URL for when we return.

Metabase (running) *BI dashboard - Days 5-6*

Same: visible, not used. The URL is on your quick-start sheet.

What is a Decision Support System?

- A Decision Support System (DSS) is a computer-based system that helps people make informed decisions - particularly for complex problems with incomplete information.
- DSS bridges the gap between raw operational data and the insight managers actually need.
- Three layers in any DSS: data collection, data transformation and storage, reporting and analysis.



This module teaches you to build the middle layer - the data warehouse - and use the right layer to visualise it.

Evolution of Decision Support Systems

1960s	MIS - Mainframe reports - fixed, periodic, batch-produced. No interactivity.
1970s	DSS - Interactive query tools. Managers could ask questions directly for the first time.
1980s	EIS - Executive dashboards. Colour displays. Pre-built KPI views for senior management.
1990s	Data Warehouse - Inmon (1990), Kimball (1996). Integrated, historical, purpose-built for analysis.
2000s+	OLAP & Self-Service BI - Slice, dice, drill-down. Excel pivot tables, then dedicated BI tools.

We are building a 1990s-style dimensional warehouse - still the dominant pattern in production today. Cloud BI wraps the same core model.

OLTP - Online Transaction Processing



Designed to record business events accurately and fast - one at a time.

- **Handles INSERT, UPDATE, DELETE - the moment something happens, it is recorded.**
- **Optimised for many short, concurrent transactions. A bank processes millions per hour.**
- **Schema is normalised (3NF): data stored once, relationships via foreign keys. Eliminates redundancy.**
- **Contains current state only - what is true right now, not what was true last year.**
- **Our spine: the NIBM enrollments table records each student-course-semester combination as it happens.**

OLTP is excellent at answering: 'What did student S001 enrol in?' It struggles with: 'How has enrolment in computing programmes changed over the past five years?'

OLAP - Online Analytical Processing



Designed to answer complex questions across large volumes of historical data.

- **Handles SELECT across millions of rows - aggregates, comparisons, trends, rankings.**
- **Optimised for read performance on large scans. Few queries, but each touches a lot of data.**
- **Schema is denormalised (star or snowflake): redundancy is acceptable because we are not writing. Redundancy here buys speed.**
- **Contains historical snapshots - we want to compare last semester to five years ago.**
- **Our spine: fact_enrollment will hold every enrollment event ever recorded, with pre-joined dimension keys.**

OLAP answers: 'How has enrolment in computing programmes changed over five years, broken down by campus?' in under two seconds.

OLTP vs OLAP - Side by Side



	OLTP	OLAP / Data Warehouse
Purpose	Record business events	Analyse historical patterns
Operation	INSERT / UPDATE / DELETE	SELECT with GROUP BY, aggregates
Schema	Normalised (3NF)	Denormalised (star schema)
Data scope	Current state only	Years of historical snapshots
Users	Applications, clerks, systems	Analysts, managers, BI tools
Query speed	Milliseconds (one row)	Seconds (millions of rows)
NIBM example	Record one enrollment	"How many enrolled per course this year?"

OLTP vs OLAP in Everyday Terms



- **OLTP is the system recording transactions right now - fast, accurate, one record at a time.**
- **OLAP is the system that looks back across all those transactions to find patterns and inform decisions.**
- **Both are essential. Neither replaces the other.**

Analogy - Cash Register vs Year-End Accountant

Familiar concept: A busy supermarket checkout till vs. the company accountant reviewing the year-end ledger.

Structural mapping: The cash register = OLTP. It records one sale at a time, fast and accurately. It cannot tell you total revenue for the year - it only knows this transaction. The accountant = OLAP. They work across the entire year's records, aggregate totals, spot trends. They do not ring individual sales - they analyse.

Where it breaks: A real accountant reads directly from the same records the till produced. In a warehouse, we copy and transform data into a separate, redesigned store. The accountant's ledger is purpose-built - that is the data warehouse.

DW reconnect: Your PostgreSQL OLTP DB is the cash register. The fact_enrollment table we build Day 2 is the ledger. The warehouse exists so the analyst never slows down the operational system.

The Four Properties of a Data Warehouse (Inmon)



Bill Inmon (1990): "A data warehouse is a subject-oriented, integrated, non-volatile, and time-variant collection of data in support of management's decision making."

Subject-oriented

Organised around business subjects - students, courses, revenue - not around processes like enrollment or payment.

Integrated

Combines data from multiple source systems. One consistent view across everything, regardless of origin.

Non-volatile

Data is loaded but rarely deleted or updated. The historical record is preserved. This is not an operational system.

Time-variant

Every record carries a time reference. The warehouse holds snapshots across years, not just what is true today.

Warehouse vs Database - What Changes



Operational Database (OLTP)

- Designed for recording transactions
- Normalised schema - avoid all redundancy
- Stores current state only
- Many small concurrent reads and writes
- Updated row-by-row as events happen
- Query: 'Is student S001 enrolled?'

Data Warehouse (OLAP)

- Designed for analysing historical patterns
- Denormalised schema - optimise for reads
- Stores historical snapshots across years
- Few large reads scanning millions of rows
- Loaded in batches (ETL) - not row-by-row
- Query: 'How has enrolment changed over 5 years?'

Warehouse vs Database in Everyday Terms

- **An operational database and a data warehouse serve different access patterns - just as two parts of a library serve different needs.**
- **Neither is better. They are architecturally different because the job is different.**

Analogy - Library Circulation vs Reference Section

Familiar concept: A university library - the circulation desk vs. the reference section.

Structural mapping: Circulation desk = operational DB: fast, transactional, one borrow or return at a time. Librarian processes and moves on - optimised for throughput, not depth. Reference section = data warehouse: you sit, browse, cross-reference across many sources, take your time. Optimised for depth of inquiry, not speed of individual transactions.

Where it breaks: A reference section holds physical books - static content. A warehouse is queried dynamically. The 'books' are the records; the query is how you navigate and combine them at runtime.

DW reconnect: DBeaver showing you the OLTP schema is the circulation desk - fast record access. The star schema we build on Day 2 is the reference section - structured for analytical inquiry. Both live in PostgreSQL. The architecture is what changes.

Discussion

What Does 'Decision Support' Actually Mean?

"Decision support" is not reporting. It means giving decision-makers the right information, in the right form, at the right time.

Think of a real decision a manager at NIBM must make. Then ask:

- What information would they need?
- Where does that information come from today?
- How long does it take to get it?
- How long should it take?

5 minutes - then share one example back with the room.

Guided Walk - DBeaver

The OLTP schema side by side with the empty DW schema

The OLTP Schema - Four Normalised Tables

students
student_id PK
first_name
last_name
email
date_of_birth
nationality

courses
course_id PK
course_name
department
credits
level

instructors
instructor_id PK
first_name
last_name
email
department

enrollments
enrollment_id PK
student_id FK
course_id FK
instructor_id FK
enrollment_date
grade
semester
academic_year

This is your DBeaver screen right now. Open the ERD view: right-click the schema → View ERD. The four tables and their FK relationships will auto-draw.

Tracing an Analytical Query Through OLTP

Business question: 'How many students enrolled in each course in Semester 1, 2024?'

```
SELECT c.course_name,  
       COUNT(e.enrollment_id)  
       AS enrolled_count  
FROM enrollments e  
JOIN students s  
  ON e.student_id = s.student_id  
JOIN courses c  
  ON e.course_id = c.course_id  
WHERE e.semester = 'Semester 1'  
      AND e.academic_year = '2024'  
GROUP BY c.course_name  
ORDER BY enrolled_count DESC;
```

1

Scan enrollments for Semester 1, 2024

2

JOIN to students (match student_id)

3

JOIN to courses (match course_id)

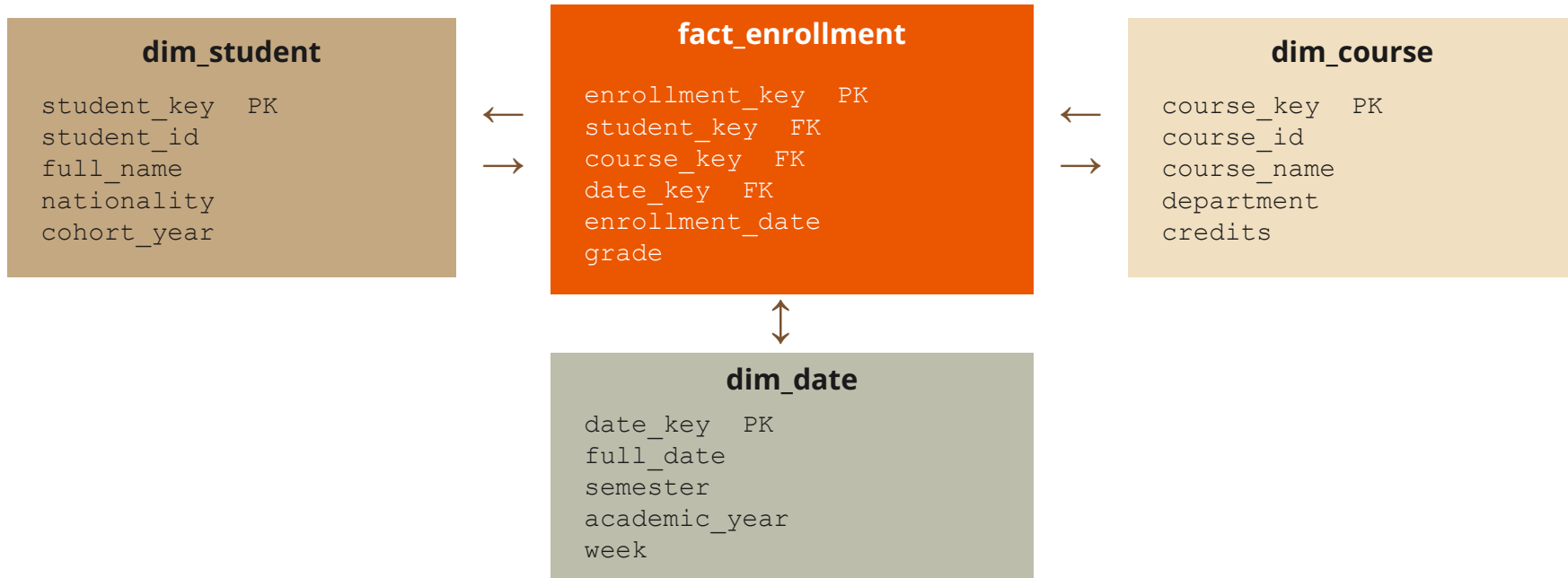
4

GROUP BY course_name, COUNT rows

Every time this query runs on growing OLTP data, all three tables are touched. The JOIN cost grows as the database grows. This is the performance problem the warehouse solves.

The Empty DW Schema - Star Shape

Alongside the OLTP schema in DBeaver, four tables exist with zero rows. This is what we fill on Day 2.



KEY CONCEPT



Denormalisation is a deliberate design choice, not a mistake.

In an operational database, redundancy is a problem - it causes update anomalies and inconsistency. In a data warehouse, **we never update individual rows**. We load, and we read. Redundancy does not cause inconsistency if the data never changes after load.

Denormalisation means pre-joining: the dimension tables store descriptive attributes so the analyst does not have to JOIN across four normalised tables every time they ask a question.





Day 1 - Lab

Querying OLTP vs DW · Requirements Sketching



Recap - Where We Are



Before we start the lab - five concepts from this morning:

- **OLTP records transactions - normalised, fast writes, current state only.**
- **OLAP analyses history - denormalised, fast reads, years of snapshots.**
- **The data warehouse is the OLAP layer: subject-oriented, integrated, non-volatile, time-variant.**
- **Denormalisation in a warehouse is a deliberate trade-off, not a mistake.**
- **Both schemas are visible in DBeaver right now - OLTP has rows; DW is empty. We fill it on Day 2.**

Our goal: answer the same business question on both systems, compare the effort, then practise turning manager requests into warehouse requirements.

Lab Activity

Same Question - Two Systems

Business question: 'How many students enrolled in each course this semester?'

Step 1 - OLTP query (DBeaver):

Run the JOIN query shown in the slides against the enrollments table. Record the row count and execution time.

Step 2 - DW query (DBeaver):

Run the star-schema query against fact_enrollment and dim_course. Note: the DW is currently empty - we will use a pre-loaded version. Record the row count and time.

Step 3: How many tables did each query touch? Which SQL is simpler to write and read?

Same Question - Two Queries Compared

OLTP Query - 3 tables, 2 JOINS

```
SELECT c.course_name,  
       COUNT(e.enrollment_id)  
       AS enrolled_count  
FROM enrollments e  
JOIN students s  
  ON e.student_id = s.student_id  
JOIN courses c  
  ON e.course_id = c.course_id  
WHERE e.semester = 'Semester 1'  
      AND e.academic_year = '2024'  
GROUP BY c.course_name  
ORDER BY enrolled_count DESC;
```

Tables touched: enrollments, students, courses

DW Query - 3 tables, 2 JOINS

```
SELECT dc.course_name,  
       COUNT(fe.enrollment_key)  
       AS enrolled_count  
FROM fact_enrollment fe  
JOIN dim_course dc  
  ON fe.course_key = dc.course_key  
JOIN dim_date dd  
  ON fe.date_key = dd.date_key  
WHERE dd.semester = 'Semester 1'  
      AND dd.academic_year = '2024'  
GROUP BY dc.course_name  
ORDER BY enrolled_count DESC;
```

Tables touched: fact_enrollment, dim_course, dim_date

Both return identical results. DW query joins pre-organised dimension tables - no need to traverse the original normalised structure on each run.

Requirements Activity

Manager Questions → DW Requirements

A manager says:

"I need to understand enrollment trends - are we growing overall? Are certain courses shrinking? Which departments are performing? I want to compare this semester to the last two years."

In pairs - 20 minutes:

1. Rewrite the manager's request as three specific, answerable business questions.
2. For each question - what data would the warehouse need? Which table? Which column?
3. Sketch the shape of the answer: would it be a number, a table, a chart?

This is your Day 1 deliverable.

Requirements Sketch - Template

Use this structure for each of your three business questions.

Business question

State it precisely. What does 'enrollment trend' mean exactly? Growth in headcount? In course completions?

Data needed

Which table(s)? Which column(s)? What time range? What grouping - by course, by department, by semester?

Shape of answer

A single number (total)? A ranked list? A time-series comparison? A breakdown by category?

Why a warehouse?

Why can't the OLTP database answer this efficiently as it grows? What gets slow or complex?

Share-Out and Day 2 Preview



Share-Out (15 minutes)

- One pair per question - present your three business questions to the room.
- What data did you identify? What shape is the answer?
- Class challenge: could the OLTP database answer it? Why or why not?

Day 2: Build It

- Dimensional modelling: facts vs dimensions, grain, star vs snowflake.
- Write the DDL for fact_enrollment + dim_student, dim_course, dim_date.
- Load seed rows and run your first analytical query.

The star schema we build on Day 2 - fact_enrollment at the centre, three dimension tables around it - is designed to answer the questions you wrote today. The empty schema in DBeaver will have data by end of Day 2.

Day 1 Deliverable



Hand in or photograph your completed sketch before leaving.

- **Scenario: NIBM enrollment data - students, courses, semesters.**
- **Three specific business questions the warehouse must answer.**
- **For each question: the data needed, the shape of the answer, and why OLTP alone is insufficient.**
- **No code required today - this is a requirements document, not a query.**

This is a rehearsal for the 50% coursework. Day 7 asks you to do the same for a new scenario - design, build, query, and visualise end-to-end.

By end of Day 1 you should be able to explain: **why a data warehouse exists, what OLTP and OLAP mean, and what the four properties of a warehouse are.**

Questions & Answers

Niranga Dharmaratna | niranga@nibm.lk

Next: Foundations (HTML Lab: OLTP vs DW comparison)

D1
Foundations

Disclaimer & Copyrights

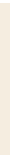


The information presented in this presentation is intended for general informational purposes only. It is not meant to be comprehensive or to constitute professional advice. Any reliance you place on such information is strictly at your own risk. While I strive to keep the information accurate and up-to-date, I make no representations or warranties of any kind, express or implied, about the completeness, accuracy, reliability, suitability, or availability concerning the content within this presentation.

This presentation may include references or links to third-party websites or content. I do not endorse the views expressed or the information provided on such external sites. I am not responsible for the content of any linked site or any link contained in a linked site.

Any action you take upon the information presented in this presentation is strictly at your own discretion. I will not be liable for any losses or damages in connection with the use of this information.

This disclaimer is subject to change without notice.



@ Images and artworks used in this presentation are used for general informational purposes only. It may contain copyrighted material, the use of which may not have been specifically authorised by the copyright owner. This material is available in an effort to explain issues relevant to the topic. This should constitute a 'fair use' of any such copyrighted material.

Last modified: 18 Jun 2026